

3D게임 프로그래밍 과제 2 설명 문서

2022184025 윤지훈

목차

1. 과제 목표
2. 가정
3. 조작법
4. 실행 결과
5. 구현 내용
6. 미흡한 점

1. 과제 목표

Wolfenstein 3D 스타일의 1인칭/3인칭 미로 슈팅 게임을 제작한다. DirectX 12의 제한된 파이프라인(입력 조립기 → 정점 셰이더 → 픽셀 셰이더 → 출력 병합기) 안에서, 절차적으로 생성된 미로를 탐험하며 등장하는 적 AI를 모두 처치하는 것을 목표로 한다. 타이틀 화면, 맵 선택 화면, 서로 다른 두 개의 맵을 모두 구현하고, 1인칭/3인칭 시점 전환, 화면 십자선 조준점, 라이프 바, 잔여 적 표시 등 기본적인 슈팅 게임 HUD를 함께 갖춘다.

2. 가정

- 0) C++20 / Windows 10 이상 / Direct3D 12 사용. 한글 코멘트는 CP949로 인코딩한다.
- 1) DirectX 12의 텍스처/지오메트리/테셀레이션 셰이더 등은 사용하지 않는다. 색상만 사용한다.
- 2) 깊이 검사는 D3D12 깊이-스텐실 버퍼가 처리한다 (D24_UNORM_S8_UINT 포맷).
- 3) 면 단위 평탄 음영(flat shading)을 사용하여 각 면을 또렷이 구분한다.
- 4) 라이팅은 단일 방향광 디퓨즈(Lambert) + 환경광. 그림자/스페큘러 없음.
- 5) 미로는 매 빌드마다 동일한 시드를 사용해 깊이 우선 탐색(DFS) 기반으로 자동 생성한다.
- 6) 적 AI는 한 번 시야에 들어온 플레이어를 영구적으로 추격한다 (시뮬레이션 단순화).
- 7) 게임은 타이틀 화면에서 시작하여 맵 선택 → 스테이지 진입 흐름을 따른다.
- 8) HUD는 NDC 좌표를 직접 사용하는 별도 셰이더로 그려, 카메라/맵 회전과 무관하게 화면에 고정된다.
- 9) 마우스 입력은 화면 중앙 캡처(SetCursorPos) 방식으로 무한 회전을 구현한다.

3. 조작법

1) 타이틀 화면

화면 중앙의 "GAME START" 버튼을 좌 클릭하면 맵 선택 화면으로 진입한다.

2) 맵 선택 화면

회전하는 두 개의 미로 미니어처 중 하나를 좌 클릭하면 해당 맵의 게임 스테이지로 진입한다.

3) 스테이지 화면

- W / A / S / D : 전후/좌우 이동
- 마우스 움직임 : 시점(Pitch/Yaw) 회전
- 마우스 좌 클릭 : 소총 발사 (쿨다운 약 0.25초)
- 스페이스 바 : 점프 (낮은 단차는 자동 점프)
- Tab 키 : 1인칭(FPS) ↔ 3인칭(TPS) 시점 전환

· ESC 키 : 마우스 캡처 해제 / 종료

4. 실행 결과

프로그램 실행 시 "THE MAZE" 타이틀 화면이 표시되고, 그 아래 회전하는 작은 미로 미리보기와 "GAME START" 버튼이 등장한다. 버튼 클릭 시 맵 선택 화면으로 전환되어 회전하는 두 개의 미니어처 미로가 나타나며, 이를 클릭하면 해당 맵의 1인칭 시점 게임이 시작된다. 게임 중에는 화면 중앙의 십자선, 좌상단의 잔여 적 카운트, 중앙 하단의 라이프 바, TPS 모드로 전환 시 어깨너머 시점의 플레이어 모델과 소총이 함께 표시된다. 모든 적을 처치하면 화면에 "WIN" 글자가 표시된 뒤 자동으로 타이틀로 복귀하고, 라이프가 0이 되면 화면이 천천히 기울며 페이드 아웃되어 타이틀 화면으로 돌아간다.

5. 구현 내용

1) 타이틀 화면 3D 텍스트와 시작 버튼 (Landing.cpp)

5×7 픽셀 격자에 알파벳을 디자인한 LetterPatternTable 을 사용해 "THE MAZE" 와 "GAME START" 두 문자열을 작은 큐브 묶음으로 표현한다. CTextLetterMesh 는 패턴의 true 셀마다 작은 큐브 정점을 쌓아 한 글자 단위 메시를 만들고, CTextStringObject 가 글자 단위 메시를 가로로 정렬해 단어를 만든다. 각 글자(CLetterObject) 는 sine 파형 wave 애니메이션 위상을 글자별로 오프셋해 부드러운 물결 효과를 만든다. CButtonObject 는 텍스트 객체를 상속하여 글자 바운딩 박스 + 패딩 영역에 화면 좌표 클릭이 들어왔는지(HitTest) 검사한다.

```
// 화면 좌표를 z=0 평면 위의 월드 좌표로 환산 - Landing.cpp::UnprojectScreenToWorld
float fNDCX = 2.0f * nMouseX / nScreenWidth - 1.0f;
float fNDCY = 1.0f - 2.0f * nMouseY / nScreenHeight;
XMMATRIX xmlnvViewProj = XMMatrixInverse(nullptr, XMMatrixMultiply(xmView, xmProj));
XMVECTOR vWorldNear = XMVector3TransformCoord(vClipNear, xmlnvViewProj);
XMVECTOR vWorldFar = XMVector3TransformCoord(vClipFar, xmlnvViewProj);
// 광선과 z=0 평면의 교점이 클릭한 월드 좌표
XMVECTOR vDir = XMVectorSubtract(vWorldFar, vWorldNear);
float t = -nearZ / (farZ - nearZ); // z=0 만나는 비율
```

2) 미로 자동 생성 — DFS + 통로 확장 + 방 배치 (Maps.cpp)

36×36 격자를 모두 벽(W)으로 채운 뒤, 깊이 우선 탐색(DFS)으로 통로(.)를 깎아 들어가며 한 줄짜리 미로 골격을 만든다. 이후 WidenCorridors() 가 25% 확률로 인접한 벽을 추가로 허물어 통로의 폭에 변화를 주고, PlantRooms() 가 2×2 ~ 4×4 크기의 직사각형 방을 6~10개 무작위로 심어 미로에 "방"의 개념을 만든다. 골격 → 폭 확장 → 방 배치 의 3단 절차가 한 줄 통로의 단조로움을 깬다.

```
// DFS 미로 골격 생성 - Maps.cpp::GenerateMaze
std::vector<std::pair<int,int>> stack;
grid[1][1] = '.'; stack.emplace_back(1, 1);
while (!stack.empty()) {
    auto [r, c] = stack.back();
    // 인접 2칸이 W인 네 방향 중 무작위 선택
    int pick = options[rng() % options.size()];
    // 사이 칸과 다음 칸을 동시에 통로로 변경
    grid[r + dirs[pick][0]/2][c + dirs[pick][1]/2] = '.';
    grid[nr][nc] = '.';
    stack.emplace_back(nr, nc);
}
```

3) 영역별 단차 부여 (PartitionFloorRegions)

보행 가능한 '.' 셀을 3~7 셀 크기의 작은 영역으로 무작위 그리디 BFS로 분할한 뒤, 각 영역마다 0~3 단계의 단차 높이를 부여한다 (한 단 = STEP_H = 0.7m). 인접 영역끼리는 가능하면 다른 높이를 선택하도록 그래프 컬러링 방식으로 보정해, 시각적으로 평탄한 덩어리가 생기지 않게 한다. 스폰 셀 (1,1) 이 속한 영역은 높이 0으로 강제하여 시작 즉시 공중에 뜨는 현상을 방지한다.

4) 벽 군집 별 높이 차등 (ComputeWallHeights)

동일한 'W' 셀로 이루어진 4방향 연결 영역(군집)을 BFS로 묶고, 군집의 크기가 10셀 이상이면 8.0/9.4/10.8/12.2 중 하나의 통일된 높이를 부여한다. 군집이 작으면 0.0을 반환하여

기본 높이 (WALL_H = 8.0)를 적용한다. "같은 한 덩어리의 벽은 같은 높이" 라는 규칙으로 자연스러운 도시형 외관을 만든다.

5) 충돌 처리와 단차 자동 점프 (GameObject.cpp::TryMoveXZ)

캐릭터의 이동은 X축과 Z축을 분리해 각각 IsBlockedInMap() 으로 검사한다. 한 축이 막혀도 다른 축은 통과시켜 벽에 비스듬히 다가갈 때 자연스럽게 미끄러진다. 막힌 셀의 윗면이 발 높이 + STEP_UP_TOLERANCE(STEP_H + 0.05)보다 낮으면 "올라설 수 있는 단차"로 판단해 통과시키고, 그보다 높으면 자동으로 점프를 시작한다. 점프 초기 속도는 $\sqrt{2gh}$ 로 계산해 정확히 단차 위에 도달할 수 있는 최소 속도를 부여한다.

```
// XZ 분리 프로브 이동 - GameObject.cpp::TryMoveXZ
if (dx != 0.0f) {
    const float probeX = pos.x + dx + (dx > 0.0f ? fRadius : -fRadius);
    if (!IsBlockedInMap(m_eSceneState, probeX, pos.z, kFeetY)) {
        pos.x += dx;
    } else {
        const float nextFloor = GetFloorHeightAt(m_eSceneState, probeX, pos.z);
        if (nextFloor > kFeetY + 0.05f) bBlockedByStep = true;
    }
}
if (bBlockedByStep && !m_bJumping && m_fJumpCooldown <= 0.0f) {
    const float jumpH = max(stepHeight - kFeetY + 0.3f, 0.5f);
    m_fVerticalVelocity = sqrtf(2.0f * kGravity * jumpH); // 단차 정확 통과
    m_bJumping = true;
}
```

6) 점프와 중력 시스템 (ApplyJumpPhysics)

스페이스 키 입력의 edge 검출(이전 프레임 상태 비교) + 점프 가능 조건 확인 후 StartJump() 으로 초기 상승 속도를 부여한다. 매 프레임 ApplyJumpPhysics() 가 중력($g = 20.0$)을 누적해 수직 속도를 감소시키고, 바닥(GetFloorHeightAt() + half-Y) 에 닿으면 m_bJumping = false 로 복귀시킨다. 단차 자동 점프와 동일한 메커니즘을 공유하여 코드 분기를 최소화했다.

7) 1인칭/3인칭 카메라 전환 + TPS 스프링암 (Camera.cpp)

CCamera 클래스가 m_fPitch / m_fYaw 두 각도를 보유하고, 이 값으로부터 look/right/up 정규직교 기저를 매 프레임 재구성한다 (RegenerateViewMatrix). FPS 모드에서는 카메라 위치가 플레이어 위치와 동일(눈높이 MAP_EYE_HEIGHT = 3.5)이지만, TPS 모드에서는 플레이어 뒤쪽으로 약 7m 떨어진 어깨너머 위치에 놓이며 SetPositionAndTarget 으로 플레이어를 정확히 바라보도록 갱신한다. TPS 카메라가 벽을 뚫고 들어가는 것을 막기 위해 ClampDistanceAgainstWalls() 가 카메라 거리 광선을 벽에 막히는 위치까지 잘라낸다 (스프링암).

```
// TPS 스프링 암 - 벽에 막히면 카메라를 그 직전까지 당긴다
const float kStep = TILE * 0.25f;
for (float d = kStep; d <= maxDist; d += kStep) {
    const float sx = fromXZ.x + dn.x * d;
    const float sz = fromXZ.z + dn.z * d;
    if (IsBlockedInMap(state, sx, sz, eyeY)) {
        const float r = d - kInset; // 벽에 살짝 떨어진 안쪽 거리
        return (r < 0.0f) ? 0.0f : r;
    }
}
```

```
}
```

8) 마우스 캡처 기반 1인칭 록 (GameFramework.cpp)

게임플레이 진입 시 `m_bMouseCaptured = true` 로 설정되어 매 프레임 마우스 좌표를 측정 후 화면 중앙으로 `SetCursorPos()` 한다. 이 방식은 마우스가 모니터 가장자리에 부딪혀 멈추는 문제 없이 무한 회전을 가능하게 한다. 화면 중앙 좌표 — 현재 좌표 = 델타 픽셀로 환산하여 카메라의 `Rotate(-fPitchDelta, fYawDelta)` 에 전달한다 (Y 부호 반전 — 마우스 위로 = 시야 위로). ESC 키나 클릭을 떼면 `m_bMouseCaptured = false` 로 캡처 해제.

9) CCharacter 베이스 클래스 (GameObject.h)

플레이어(CPlayer)와 적(CEnemyObject)이 공통으로 가지는 체력, 점프 물리, 소총 부착, XZ 분리 프로브 이동, 점프 쿨다운 등의 기능을 CCharacter 라는 중간 클래스로 추출했다. 서브클래스는 차별화되는 부분만 오버라이드: CPlayer 는 입력 처리와 반동 타이머, CEnemyObject 는 AI 상태 머신과 A* 길찾기. 이 추상화 덕분에 적 추가/수정 시 코드 중복이 거의 없다.

10) 적 AI — 상태머신 + LOS + A* 길찾기 (CEnemyObject, Maps.cpp)

적은 `Idle_Move`(패트롤 이동) / `Idle_Pause`(정지) / `Pursue`(추적) 세 가지 상태를 가지며, 매 프레임 `HasLineOfSight()` 로 플레이어와의 시야를 검사한다. 한 번이라도 시야에 잡히면 `m_bAware = true` 가 되어 영구적으로 `Pursue` 상태로 전환된다. `Pursue` 상태에서는 `FindPathAStar()` 로 그리드 셀 단위 최단 경로를 탐색하고, 0.5초마다 경로를 재계산하면서 다음 웨이포인트로 이동한다. 발사 쿨다운이 끝나면 총알을 발사하고 약 0.3초간 정지하여 "조준" 연출을 보인다.

```
// A* 길찾기 - Maps.cpp::FindPathAStar
// 'W' 셀만 불통, 단차는 통과 가능 (TryMoveXZ 점프가 처리)
auto isWalkable = [&](int c, int r) {
    return (c>=0 && c<cols && r>=0 && r<rows) ? (grid[r][c] != 'W') : false;
};
auto heuristic = [&](int c, int r) {
    return float(abs(c - goalC) + abs(r - goalR)) * TILE; // 맨해튼
};
while (!openSet.empty() && nodesExpanded < nMaxNodes) {
    int curIdx = openSet.top().second; openSet.pop();
    if (curIdx == goalIdx) { /* 부모 추적으로 경로 복원 */ }
    for (int i = 0; i < 4; ++i) {
        float newG = gCost[curIdx] + TILE;
        if (newG < gCost[nIdx]) {
            gCost[nIdx] = newG; parent[nIdx] = curIdx;
            openSet.emplace(newG + heuristic(nc, nr), nIdx);
        }
    }
}
```

11) 적 스폰 시스템 (PickEnemySpawnPositions)

현재 맵에서 적이 스폰 가능한 위치(평면/단차 바닥) 중 플레이어의 시작 타일과 Chebyshev 거리 ≥ 5 인 셀만 후보로 삼고, `std::shuffle` 로 섞은 뒤 최대 `nMax` 개 추출한다. Chebyshev 거리(행/열 max) 기준은 "한 타일 거리 = 4m" 환산으로 약 20m 떨어진 곳부터 스폰되어 즉시 사격당하지 않도록 한다. 반환되는 `XMFLOAT3` 의 Y 는 해당 타일의 바닥 + `halfBodyY` 로

설정되어 있어 그대로 적 위치로 사용 가능하다.

12) 사격 시스템 — 광선 추적 조준 + 반동 (GameFramework.cpp)

FireBullet() 은 화면 중앙(십자선) 방향으로 광선을 쏘 GetAimTargetPoint() 로 첫 충돌점을 계산한다. 광선은 우선 살아있는 적의 AABB와 슬랩-라인 교차를 검사하고, 그 다음 벽 셀과 step-march 검사를 한다. 총구는 소총 메시의 +Z 축에서 1.25 단위 앞에 위치하며, 조준 대상점을 향해 정확히 발사된다. 발사 직후 m_fRecoilTimer 가 0으로 리셋되어 3-키프레임 반동 애니메이션(뒤로 후진 → 절반 복귀 → 원위치)이 시작된다.

```
// 조준 광선 - 적 AABB 우선, 벽 fallback - GameFramework.cpp
XMFL0AT3 CGameFramework::GetAimTargetPoint() const {
    auto enemyAABBs = m_pScene->GetAliveEnemyAABBs();
    float bestT = kMaxAimDist;
    for (auto& [center, half] : enemyAABBs) {
        // 슬랩-라인(Slab) 교차 테스트로 광선의 가까운 t값 산출
        if (RayAABBIntersect(origin, look, center, half, t) && t < bestT)
            bestT = t;
    }
    // 벽 step-march 로 더 가까운 충돌점이 있는지 확인
    float wallT = ClampDistanceAgainstWalls(state, origin, look, kMaxAimDist, eyeY);
    return origin + look * min(bestT, wallT);
}
```

13) 일반화된 충돌 시스템 (Collision.h, EObjectTag)

객체마다 EObjectTag enum 값(Generic / Bullet / Enemy / Player / Wall / Pickup / EnemyBullet) 을 부여하고, Collision::CheckCollisions(objects, tagA, tagB, callback) 으로 두 태그 쌍의 모든 AABB 겹침을 검사한다. Bullet vs Enemy / EnemyBullet vs Player 두 쌍이 활성화되어 있으며, Bullet vs EnemyBullet 처럼 등록하지 않은 쌍은 서로 통과한다. 새 객체 타입 추가 시 새 태그 값과 한 줄의 콜백 등록만으로 충돌 페어를 확장할 수 있다.

```
// 일반화된 충돌 검사 - Scene.cpp::AnimateObjects
Collision::CheckCollisions(
    m_vShaders[idx].GetObjects(),
    EObjectTag::Bullet, EObjectTag::Enemy,
    [](CGameObject* a, CGameObject* b) {
        auto* pEnemy = static_cast<CEnemyObject*>(b);
        if (pEnemy->IsDying()) return;
        a->Kill(); // 총알 소멸
        b->OnHit(a); // 적 HP 감소 → 0이면 사망 애니메이션 진입
    });
```

14) HUD 시스템 — 십자선 / 라이프바 / 적 카운트 / WIN / 비네트

HUD 요소는 모두 CHudShader 를 사용하며, 정점이 이미 NDC([-1,1]) 좌표로 저장되어 있어 VSHud 셰이더가 월드/뷰/투영을 거치지 않고 그대로 출력한다. 깊이 테스트와 컬링을 모두 OFF 로 두어 어떤 화면 위치든 항상 그려진다. CCrosshairMesh(십자선), CLifeBarMesh(라이프 바), CHudQuadMesh(적 잔여 카운트의 점 + "WIN" 글자 세그먼트) 가 모두 같은 셰이더를 공유한다. 피격 시에는 CHitOverlayShader 가 알파 블렌딩이 켜진 별도 PSO 로 풀스크린 쿼드를 그리고, PS에서 화면 중앙으로부터의 거리에 따라 smoothstep 으로 외곽 비네트를 만든다.

```
// HLSL - 화면 외곽 피격 비네트 - Shaders.hlsl::PSHitOverlay
float4 PSHitOverlay(VS_OVERLAY_OUTPUT input) : SV_TARGET {
    // |Indc| 의 코너값(√2) 을 1.0 으로 정규화해 외곽 가중치 계산
```



```

float r = length(input.ndc) * 0.7071f;
// 중심부 ~45% 반경은 완전 투명, 외곽으로 갈수록 부드럽게 짙어진다
float mask = smoothstep(0.45f, 1.0f, r);
float a = mask * gHitFlash;
if (a < 0.005f) discard; // 거의 안 보이면 일찍 종료
return float4(0.85f, 0.05f, 0.05f, a);
}

```

15) 적/플레이어 사망 애니메이션 (GameObject.cpp)

적이 HP 0에 도달하면 즉시 Kill() 하지 않고 m_fDeathTimer 를 설정해 "죽는 중" 상태로 진입한다. Animate() 분기에서 0.2초 정지 후 0.8초 동안 사망 시점의 forward 축을 중심으로 90° 회전시켜 옆으로 쓰러뜨린다. 동시에 Y 좌표를 0.7m 만큼 lerp 로 하강시켜 회전으로 줄어든 수직 AABB 만큼 바닥에 자연스럽게 안착시킨다. 사망 시점의 소총 위치를 본체-로컬 좌표로 캐시한 뒤 ($R_{local} = R_{world} \times inverse(B_{world})$) 매 프레임 새 본체 행렬에 곱해 소총이 본체에 강체 부착된 것처럼 같이 회전·하강하도록 한다. 플레이어 사망은 m_fPlayerDeathTimer 로 1.5초 동안 카메라가 천천히 앞으로 기울며 입력이 차단되고, 만료 시 LANDING 화면으로 자동 복귀한다.

```

// 사망 시점 소총의 본체-로컬 행렬 캐시 - GameObject.cpp::CEnemyObject::OnHit
if (m_pRifle) {
    XMMATRIX matRifleWorld = XMLoadFloat4x4(&m_pRifle->GetWorldMatrixRef());
    XMMATRIX matBodyWorld = XMLoadFloat4x4(&m_xmf4x4World);
    XMVECTOR vDet;
    XMMATRIX matBodyInv = XMMatrixInverse(&vDet, matBodyWorld);
    XMStoreFloat4x4(&m_xmf4x4DeathRifleLocal,
        XMMatrixMultiply(matRifleWorld, matBodyInv));
    // 이후 매 프레임: R_world(t) = R_local * B_world(t) 로 복원
}

```

16) 디퓨즈 라이팅 + 면 단위 평탄 음영 (Shaders.hlsl, Mesh.cpp)

단일 방향광 디퓨즈(Lambert) + 환경광 모델로 라이팅을 구현한다. cbLightInfo(b2) 에 정규화된 광선 방향과 색상, 환경광 색상을 PS 전용 root constant로 업로드한다. 면 단위 평탄 음영(flat shading)을 위해 큐브 메시의 정점을 8개 → 24개로 분리하여 각 면이 자신의 노멀($\pm X / \pm Y / \pm Z$)을 갖도록 한다. 노멀은 위치/색 정점 버퍼와 분리된 slot 1 의 별도 정점 버퍼로 바인딩한다 (CNormalVertex).

```

// HLSL - Lambert + 환경광 - Shaders.hlsl::PSDiffused
float4 PSDiffused(VS_OUTPUT input) : SV_TARGET {
    float3 N = normalize(input.normalWorld); // 보관된 노멀 재정규화
    float3 L = normalize(-gLightDirWorld); // 라이트 방향의 역
    float ndotL = saturate(dot(N, L));
    float3 lit = input.color.rgb * (gAmbientColor + gLightColor * ndotL);
    return float4(lit, input.color.a);
}

```

17) 정적 미로 메시 통합 — 드로우 콜 최적화 (CMergedCubeMesh)

초기 구현에서 미로의 각 큐브($36 \times 36 \times 2$ 2592개)를 개별 GameObject 로 그리는 방식은 매 프레임 수천 번의 draw call을 발생시켜 CPU 바운드 병목을 만들었다. 회전이 없는 정적 큐브의 정점을 월드 좌표로 미리 변환해 단일 정점/인덱스 버퍼로 통합하고, 한 번의 DrawIndexedInstanced 로 모든 벽/바닥을 그리도록 변경했다. 결과적으로 1800+ draw call이 1개로 줄어 GPU 부하와 CPU 명령 전송이 모두 극적으로 감소했다. 동적 객체(적/총알)는

그대로 개별 객체로 유지한다.

18) Wavefront .obj 파일 로딩 (CObjMesh)

소총 모델(M16) 을 외부 .obj 파일로부터 로드한다. v / vn / f / o 토큰을 직접 파싱하고, n-gon(>3) 페이스는 fan triangulation 으로 삼각형으로 분할한다. vn 이 없는 페이스는 정점 위치로부터 외적으로 면 노멀을 자동 계산한다. 모델 좌표계를 엔진 좌표계(LH, +Z=forward)로 변환하기 위해 호출자가 변환 행렬을 주입하면, 정점/노멀에 미리 베이킹한다. o 그룹 이름을 이용해 "Stock" → 우드, "Mag" → 짙은 메탈 등 부품별 색상을 자동 분배한다.

19) 씬 분리와 전이 + 미니어처 미리보기 (Scene.cpp)

CScene 클래스는 SceneState $\in$ {LANDING, MAP_SELECT, MAP1, MAP2} 의 enum 으로 현재 활성 씬을 구분한다. 각 씬은 자신만의 CObjectsShader 와 GameObject 목록을 m_vShaders 벡터의 슬롯에 보유하며, Render() 는 현재 상태 슬롯만 그린다. LANDING 에서 "GAME START" 클릭 → MAP_SELECT 로 전이, MAP_SELECT 에서 미니어처 클릭 → MAP1 또는 MAP2 로 전이, 모든 적 처치 또는 사망 시 → LANDING 으로 복귀하는 식의 전이를 TransitionToScene() 으로 처리한다.

맵 선택 화면에서는 두 개의 게임 맵을 그대로 작은 크기(scale 0.25 / 0.18)로 회전시키며 보여준다. RenderInParent() 가 자식 객체의 월드 행렬에 부모 변환(축소 + X축 45° 기울임 + Y축 회전 + 좌/우 이동)을 곱해 동일한 미로 메시를 미니어처로 재활용한다. 마우스 호버 시 hover scale 1.18 을 추가 적용해 선택 가능 영역을 시각적으로 강조한다. 타이틀 화면에서도 같은 방식으로 MAP1 을 화면 우하단에 축소 표시한다.

20) 잔여 적 마커 + 승리 "WIN" 글자

잔여 적이 3 마리 이하가 되면 Scene::SetEnemyMarkersVisible(true) 가 호출되어 살아있는 적의 머리 위에 노란 기둥 마커가 표시된다. 마커는 적과 별도의 CGameObject 로 매 프레임 위치만 동기화하며, 메시 길이를 14m 로 키워 8m 높이의 벽 너머로도 보인다. 모든 적을 처치하면 m_fVictoryTimer 가 시작되고 화면 중앙에 "WIN" 글자가 CHudQuadMesh 세그먼트 조합(W=4 stroke, l=3 stroke, N=3 stroke)으로 표시된 뒤, 타이머 만료 시 LANDING 으로 자동 복귀한다.

6. 미흡한 점

- 1) 적의 총알 회피 행동이 없어 플레이어가 자리만 잡으면 적이 쉽게 제거된다.
- 2) 적과 적 사이의 충돌 처리가 없어 같은 좁은 통로에 여러 적이 겹쳐 들어갈 수 있다.
- 3) 게임 사운드(BGM, 발사음, 피격음)가 없다.
- 4) 텍스처를 사용할 수 없어 큐브 색상만으로 미로의 시각적 다양성을 표현해야 했다.
- 5) A* 길찾기가 모든 적에 대해 0.5초마다 재계산되므로 적이 많아지면 CPU 부하가 누적될 수 있다.
- 6) 1인칭 모드에서 자신의 소총만 보이고 손/팔이 렌더링되지 않는다.
- 7) 사망 시 "GAME OVER" 같은 명시적 텍스트 표시 없이 페이드만 적용되어 처음 보면 종료된 줄 모를 수 있다.